

1.charAt() — courseName :

Objective : To understand and demonstrate the Java String method charAt() using a simple example.

Task Description: Write and execute the class CharacterAtDemo to demonstrate that charAt() returns the character at a specified index position

Steps

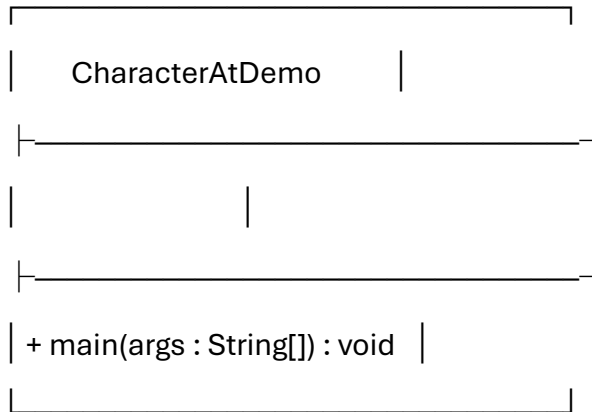
1. Create a Java class named CharacterAtDemo.
2. Define the main() method inside the class.
3. Declare a String variable named courseName.
4. Assign "java programming" to the variable.
5. Print the value of courseName.
6. Use the charAt(1) method to access the character at index 1.
7. Print the character returned by charAt(1).
8. Compile and run the Java program.
9. Observe the output.

Algorithm

Algorithm: Find the character at a specific index in a String

1. **Start**
2. Define a class named CharacterAtDemo.
3. Start the main() method.
4. Declare a String variable courseName and assign "java programming" to it.
5. Display the course name.
6. Use the charAt(1) method to get the character at **index 1**.
7. Display the character obtained.
8. **Stop**

2.UML DIAGRAM:



Code :

```
class CharacterAtDemo {  
    public static void main(String[] args) {  
        String courseName = "java programming";  
  
        System.out.println("Course: " + courseName);  
        System.out.println("Character at index 1: " + courseName.charAt(1));  
    }  
}
```

Output :

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'CharacterAtDemo'  
Course: java programming  
Character at index 1: o  
PS C:\Users\yabhi\abhi.java>
```

The screenshot shows a terminal window with the command prompt. The command executed is `PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'CharacterAtDemo'`. The output of the program is displayed below the command: `Course: java programming` and `Character at index 1: o`. The prompt returns to `PS C:\Users\yabhi\abhi.java>`.

StartsWith() —

Objective: To understand and demonstrate the Java String method startsWith() using a simple example.

Task Description : Write and execute the class StartsWithDemo to demonstrate that startsWith() checks whether a string begins with a specified prefix

Steps

1. Start the program.
 2. Create a class named StartsWithDemo.
 3. Define the main() method.
 4. Create a String variable websiteName.
 5. Assign "JavaProgramming" to websiteName.
 6. Use the startsWith("Java") method to check whether the string starts with "Java".
 7. Display the result using System.out.println().
 8. Stop the program.
-

2. Algorithm

Algorithm: To check whether a string starts with "Java"

1. **Start**
2. Declare a String variable websiteName.
3. Assign "JavaProgramming" to websiteName.
4. Apply the startsWith("Java") method to websiteName.
5. Obtain the Boolean result (true or false).
6. Display "Starts with Java: " followed by the result.
7. **Stop**

UML Diagram :

+-----+

| **StartsWithDemo** |

+-----+

| |

+-----+

| + main(String[] args): void |

+-----+

|

| **creates**

↓

+-----+

| **String** |

+-----+

| **websiteName = "JavaProgramming"** |

+-----+

|

| **calls**

↓

+-----+

| **startsWith("Java")** |

+-----+

|

| **returns**

↓

true / false

|

↓

System.out.println()

Code :

```
class StartsWithDemo {  
    public static void main(String[] args) {  
        String websiteName = "JavaProgramming";  
  
        System.out.println("Starts with Java: "  
            + websiteName.startsWith("Java"));  
    }  
}
```

Output :



The screenshot shows a terminal window with a dark background. At the top, there is a tab bar with 'Problems 32', 'Output', 'Debug Console', 'Terminal' (which is selected and highlighted with a yellow border), and 'Ports'. The terminal content shows a PowerShell prompt 'PS C:\Users\yabhi\abhi.java>' followed by a command to run a Java program. The command is: `& 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StartsWithDemo'`. The output of the program is 'Starts with Java: true'. The prompt returns to 'PS C:\Users\yabhi\abhi.java>'.

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StartsWithDemo'  
Starts with Java: true  
PS C:\Users\yabhi\abhi.java>
```

EndsWith() —

Objective : To understand and demonstrate the Java String method `endsWith()` using a simple example.

Task Description: Write and execute the class `EndsWithDemo` to demonstrate that `endsWith()` checks whether a string ends with a specified suffix

Steps

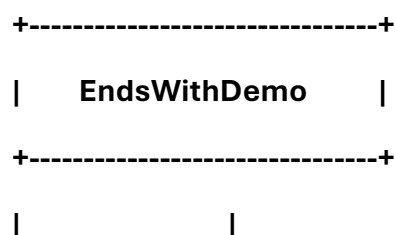
1. Start the program.
 2. Create a class named `EndsWithDemo`.
 3. Define the `main()` method.
 4. Create a String variable `fileName` and assign `"Student.java"`.
 5. Use the `endsWith(".java")` method to check whether the filename ends with `.java`.
 6. Display the result using `System.out.println()`.
 7. Stop the program.
-

2. Algorithm

Algorithm: To check whether a filename ends with `.java`

1. Start
2. Declare a String variable `fileName`.
3. Assign `"Student.java"` to `fileName`.
4. Apply the `endsWith(".java")` method to `fileName`.
5. Store/obtain the Boolean result (true or false).
6. Display `"Ends with .java: "` followed by the result.
7. Stop

UML Diagram :



```

+-----+
| + main(String[] args): void |
+-----+

|
| creates
v
+-----+
| String |
+-----+

| fileName = "Student.java" |
+-----+

|
| calls
v
+-----+
| endsWith(".java") |
+-----+

|
| returns
v
true / false

|
v
System.out.println()

```

Output :

```

Problems 32  Output  Debug Console  Terminal  Ports
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'EndsWithDemo'
Ends with .java: true
PS C:\Users\yabhi\abhi.java>

```

length() —

Objective: To understand and demonstrate the Java String method `length()` using a simple example.

Task Description: Write and execute the class `StringLengthDemo` to demonstrate that `length()` finds the number of characters in a string

Steps

1. Start the program.
 2. Create a class named `StringLengthDemo`.
 3. Define the `main()` method.
 4. Create a `String` variable `studentName`.
 5. Assign "Abhi" to `studentName`.
 6. Display the student name using `System.out.println()`.
 7. Use the `length()` method to find the number of characters in the string.
 8. Display the length of the string.
 9. Stop the program.
-

2. Algorithm

Algorithm: To find the length of a String

1. Start
 2. Declare a `String` variable `studentName`.
 3. Assign "Abhi" to `studentName`.
 4. Display the value of `studentName`.
 5. Apply the `length()` method to `studentName`.
 6. Display the length of the String.
 7. Stop
-

3. UML Diagram

```
+-----+
|      StringLengthDemo      |
```

```

+-----+
|           |
+-----+
| + main(String[] args): void |
+-----+
|
| creates
↓
+-----+
|      String      |
+-----+
| studentName = "Abhi" |
+-----+
|
| calls
↓
+-----+
|    length()    |
+-----+
|
| returns length
↓
System.out.println()

```

Output :

```

problems 32  Output  Debug Console  Terminal  Ports
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StringLengthDemo'
Name: Abhi
Length: 4
PS C:\Users\yabhi\abhi.java>

```

ToUpperCase() —

Objective: To understand and demonstrate the Java String method toUpperCase() using a simple example.

Task Description: Write and execute the class UpperCaseDemo to demonstrate that toUpperCase() converts all letters in a string to uppercase.

Steps

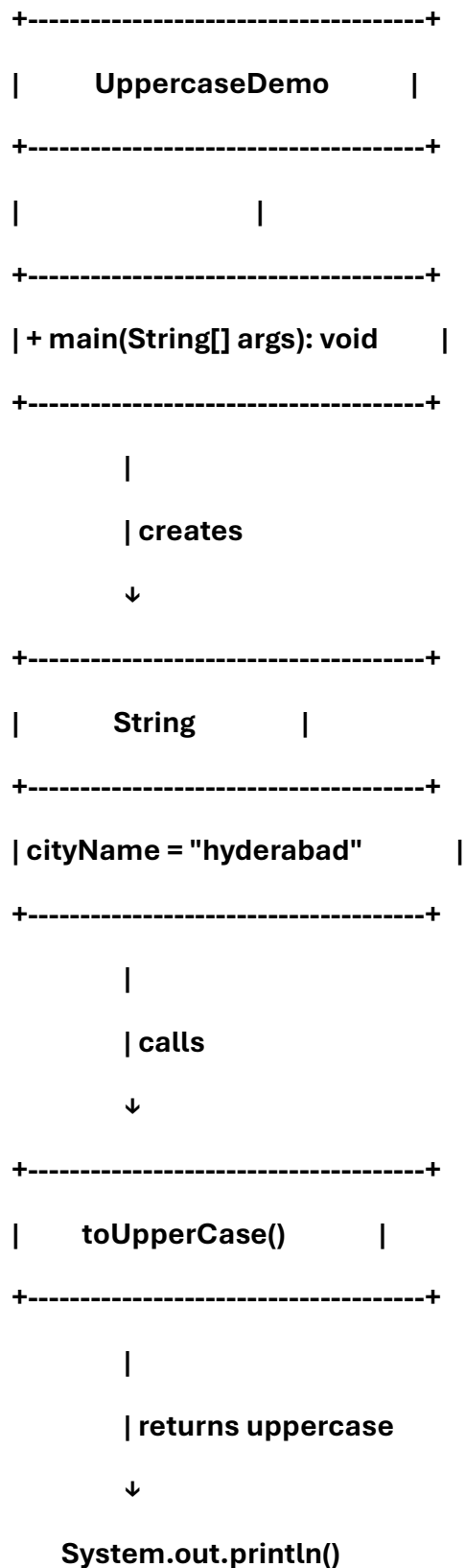
1. Start the program.
 2. Create a class named UppercaseDemo.
 3. Define the main() method.
 4. Create a String variable cityName.
 5. Assign "hyderabad" to cityName.
 6. Display the original city name.
 7. Use the toUpperCase() method to convert the string into uppercase letters.
 8. Display the uppercase city name.
 9. Stop the program.
-

2. Algorithm

Algorithm: To convert a String into uppercase

1. Start
 2. Declare a String variable cityName.
 3. Assign "hyderabad" to cityName.
 4. Display the original value of cityName.
 5. Apply the toUpperCase() method to cityName.
 6. Display the converted uppercase String.
 7. Stop
-

3. UML Diagram



Code :

```
class UppercaseDemo {  
    public static void main(String[] args) {
```

```
String cityName = "hyderabad";

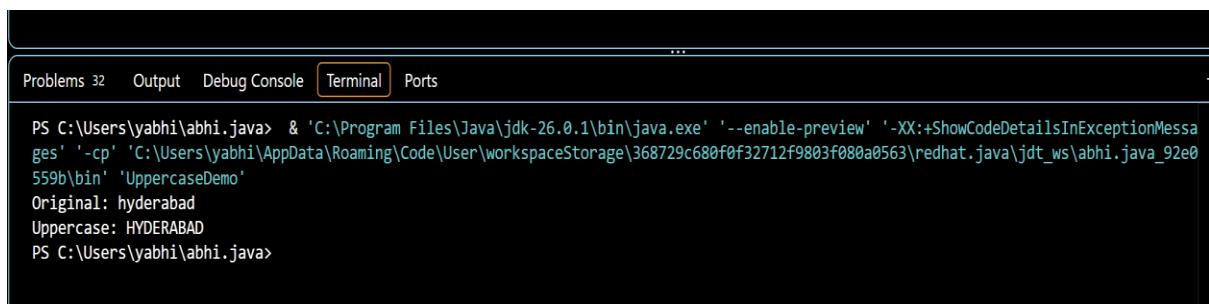
System.out.println("Original: " + cityName);

System.out.println("Uppercase: " + cityName.toUpperCase());

}

}
```

Output :

A screenshot of a Java IDE's terminal window. The terminal has tabs for 'Problems 32', 'Output', 'Debug Console', 'Terminal' (which is selected and highlighted with an orange border), and 'Ports'. The terminal text shows the command to run a Java program: 'PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'UppercaseDemo''. The output of the program is displayed below the command: 'Original: hyderabad' and 'Uppercase: HYDERABAD'. The prompt 'PS C:\Users\yabhi\abhi.java>' is shown at the bottom of the terminal.

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'UppercaseDemo'
Original: hyderabad
Uppercase: HYDERABAD
PS C:\Users\yabhi\abhi.java>
```

ToLowerCase() :

Objective: To understand and demonstrate the Java String method toLowerCase() using a simple example.

Task Description: Write and execute the class LowerCaseDemo to demonstrate that toLowerCase() converts all letters in a string to lowercase

Steps

1. Start the program.
 2. Create a class named LowerCaseDemo.
 3. Define the main() method.
 4. Create a String variable collegeName.
 5. Assign "ENGINEERING COLLEGE" to collegeName.
 6. Display the original college name.
 7. Use the toLowerCase() method to convert the string into lowercase letters.
 8. Display the lowercase college name.
 9. Stop the program.
-

2. Algorithm

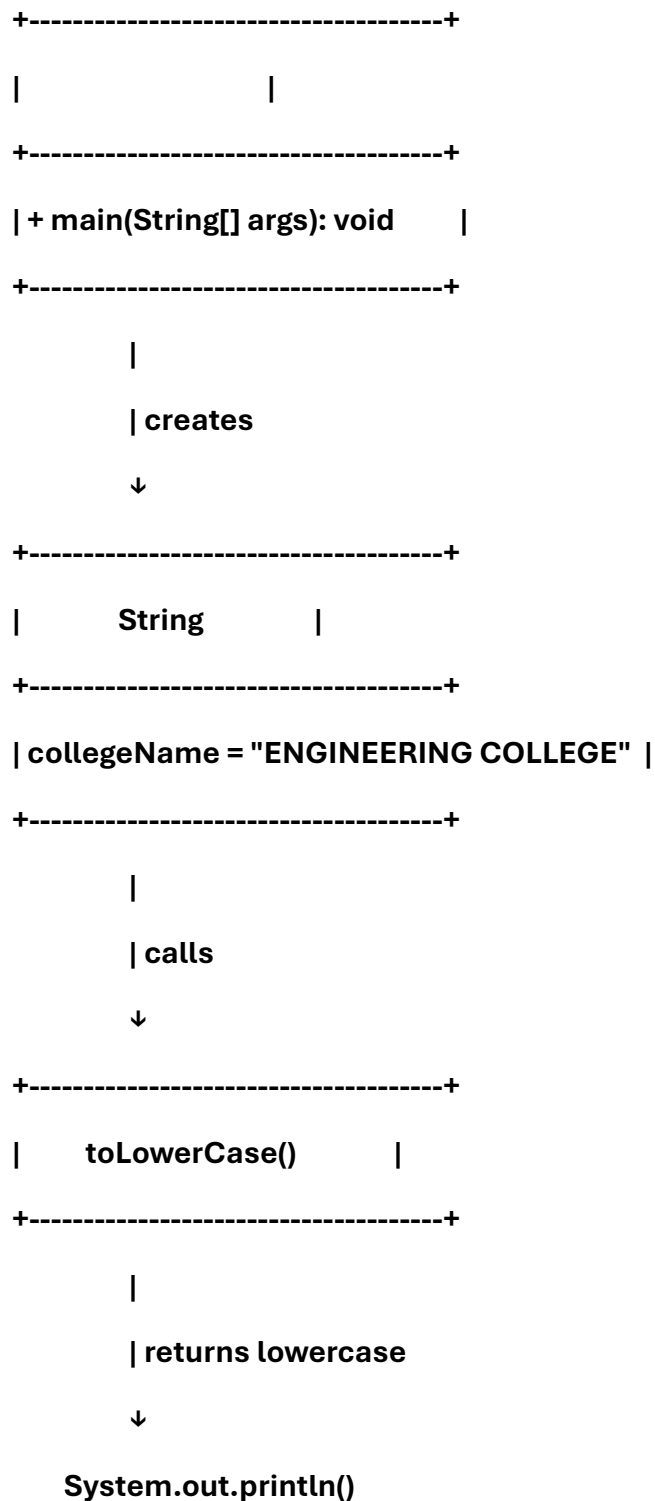
Algorithm: To convert a String into lowercase

1. Start
 2. Declare a String variable collegeName.
 3. Assign "ENGINEERING COLLEGE" to collegeName.
 4. Display the original value of collegeName.
 5. Apply the toLowerCase() method to collegeName.
 6. Display the converted lowercase String.
 7. Stop
-

3. UML Diagram

+-----+

| LowerCaseDemo |



Code :

```
class LowerCaseDemo {  
    public static void main(String[] args) {  
        String collegeName = "ENGINEERING COLLEGE";
```

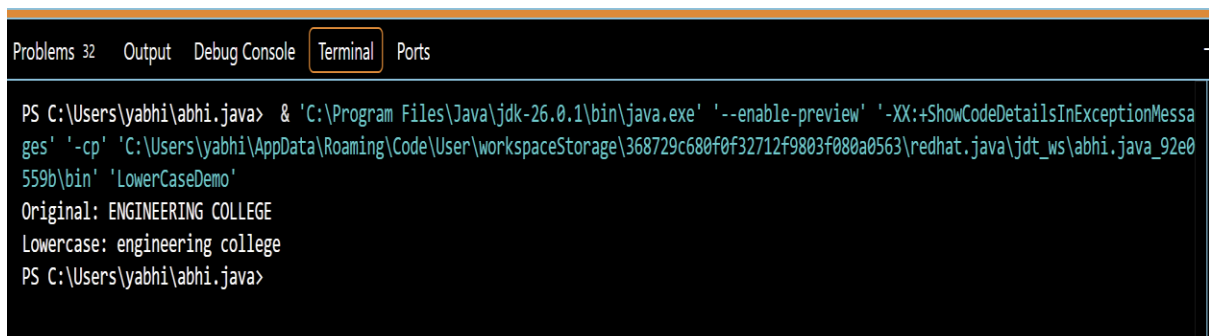
```
        System.out.println("Original: " + collegeName);

        System.out.println("Lowercase: " + collegeName.toLowerCase());

    }

}
```

Output :

A screenshot of a Java IDE's terminal window. The terminal has tabs for 'Problems 32', 'Output', 'Debug Console', 'Terminal' (which is active), and 'Ports'. The command prompt shows the execution of a Java program named 'LowerCaseDemo'. The output of the program is displayed in the terminal, showing the original string 'ENGINEERING COLLEGE' and its lowercase version 'engineering college'.

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'LowerCaseDemo'
Original: ENGINEERING COLLEGE
Lowercase: engineering college
PS C:\Users\yabhi\abhi.java>
```

Equals() —

Objective: To understand and demonstrate the Java String method equals() using a simple example.

Task Description: Write and execute the class StringEqualsDemo to demonstrate that equals() compares two strings and returns true when their contents are equal.

Steps

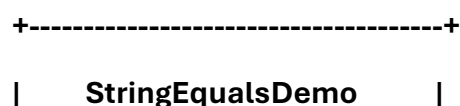
1. Start the program.
 2. Create a class named StringEqualsDemo.
 3. Define the main() method.
 4. Create a String variable firstWord and assign "Java".
 5. Create another String variable secondWord and assign "Java".
 6. Use the equals() method to compare firstWord and secondWord.
 7. Display the comparison result using System.out.println().
 8. Stop the program.
-

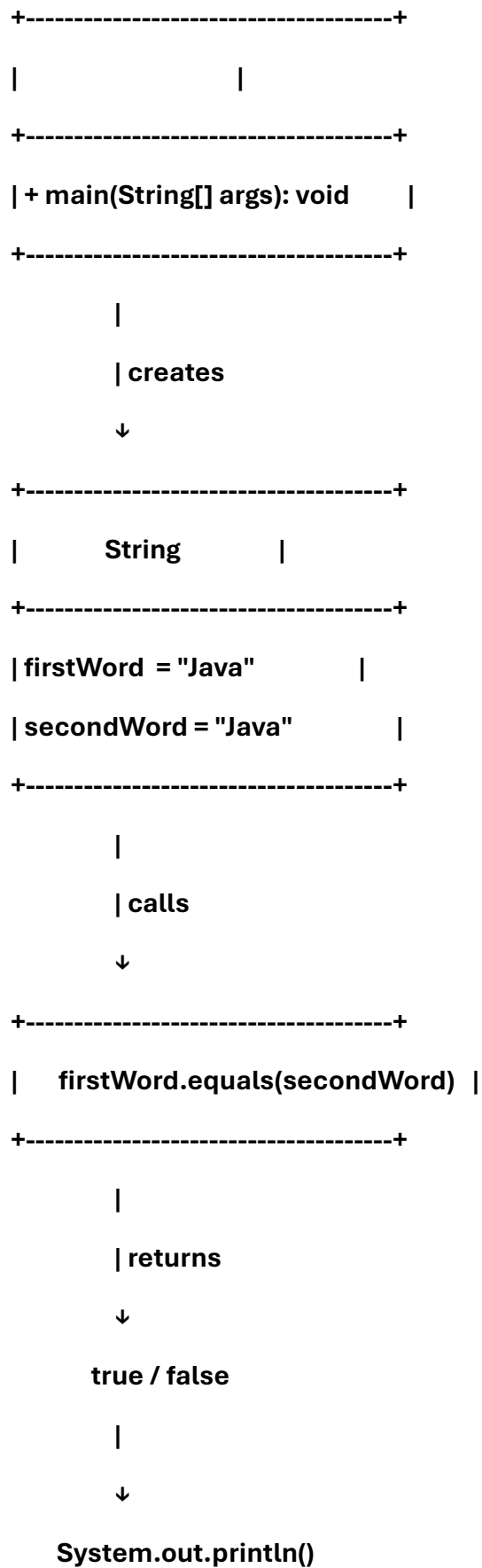
2. Algorithm

Algorithm: To compare two Strings

1. Start
 2. Declare a String variable firstWord.
 3. Assign "Java" to firstWord.
 4. Declare another String variable secondWord.
 5. Assign "Java" to secondWord.
 6. Apply the equals() method to compare firstWord with secondWord.
 7. Display the result of the comparison.
 8. Stop
-

3. UML Diagram

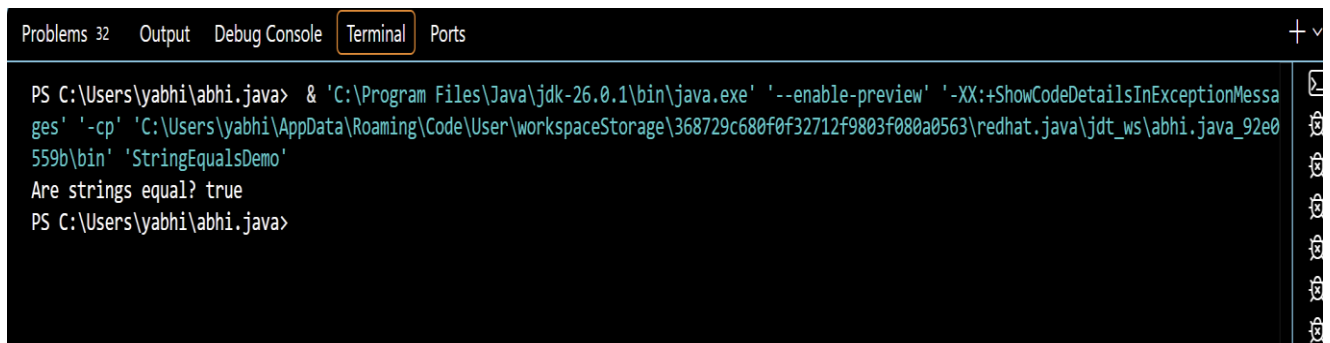




Code :

```
class StringEqualsDemo {  
  
    public static void main(String[] args) {  
  
        String firstWord = "Java";  
  
        String secondWord = "Java";  
  
        System.out.println("Are strings equal? " + firstWord.equals(secondWord));  
  
    }  
}
```

Output :



The screenshot shows a terminal window with a dark background. The title bar at the top includes tabs for 'Problems 32', 'Output', 'Debug Console', 'Terminal' (which is selected and highlighted with a yellow border), and 'Ports'. The terminal content shows a PowerShell prompt 'PS C:\Users\yabhi\abhi.java>' followed by a command to run a Java program. The command is: `& 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StringEqualsDemo'`. The output of the program is 'Are strings equal? true'. The prompt returns to 'PS C:\Users\yabhi\abhi.java>'.

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StringEqualsDemo'  
Are strings equal? true  
PS C:\Users\yabhi\abhi.java>
```

EqualsIgnoreCase() —

Objective: To understand and demonstrate the Java String method `equalsIgnoreCase()` using a simple example.

Task Description: Write and execute the class `IgnoreCaseDemo` to demonstrate that `equalsIgnoreCase()` compares two strings without considering uppercase or lowercase differences.

Steps

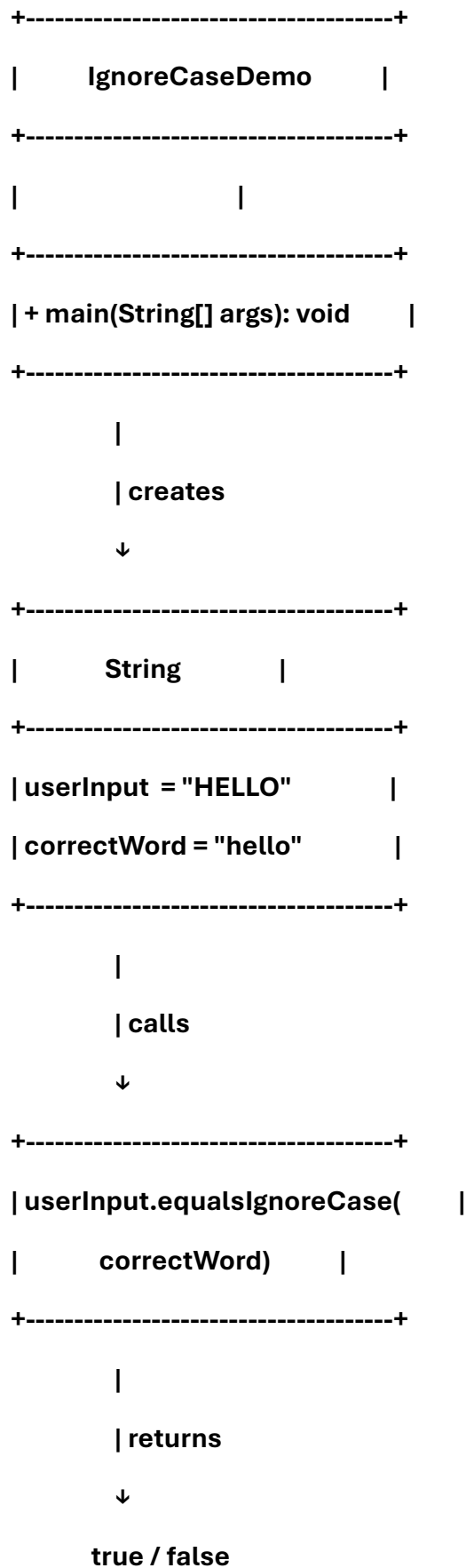
1. Start the program.
 2. Create a class named `IgnoreCaseDemo`.
 3. Define the `main()` method.
 4. Create a String variable `userInput` and assign "HELLO".
 5. Create another String variable `correctWord` and assign "hello".
 6. Use the `equalsIgnoreCase()` method to compare both strings without considering uppercase or lowercase letters.
 7. Display the comparison result using `System.out.println()`.
 8. Stop the program.
-

2. Algorithm

Algorithm: To compare two Strings ignoring case

1. Start
 2. Declare a String variable `userInput`.
 3. Assign "HELLO" to `userInput`.
 4. Declare a String variable `correctWord`.
 5. Assign "hello" to `correctWord`.
 6. Apply the `equalsIgnoreCase()` method to compare both strings.
 7. Ignore the difference between uppercase and lowercase letters.
 8. Display the comparison result.
 9. Stop
-

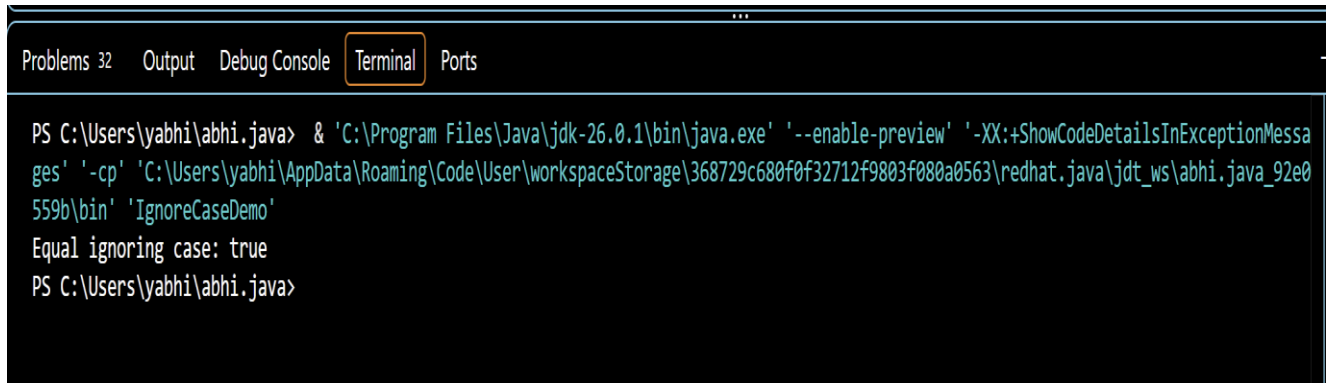
3. UML Diagram



|
↓

System.out.println()

Output :



The screenshot shows an IDE terminal window with a dark background. At the top, there is a tab bar with four tabs: 'Problems 32', 'Output', 'Debug Console', and 'Terminal'. The 'Terminal' tab is selected and highlighted with an orange border. Below the tab bar, the terminal displays the following text:

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'IgnoreCaseDemo'  
Equal ignoring case: true  
PS C:\Users\yabhi\abhi.java>
```

Contains() —

Objective: To understand and demonstrate the Java String method contains() using a simple example.

Task Description: Write and execute the class StringContainsDemo to demonstrate that contains() checks whether a specified sequence of characters exists in a string.

Steps

1. Start the program.
 2. Create a class named StringContainsDemo.
 3. Define the main() method.
 4. Create a String variable messageText.
 5. Assign "Java is very easy" to messageText.
 6. Use the contains() method to check whether the string contains "easy".
 7. Display the result using System.out.println().
 8. Stop the program.
-

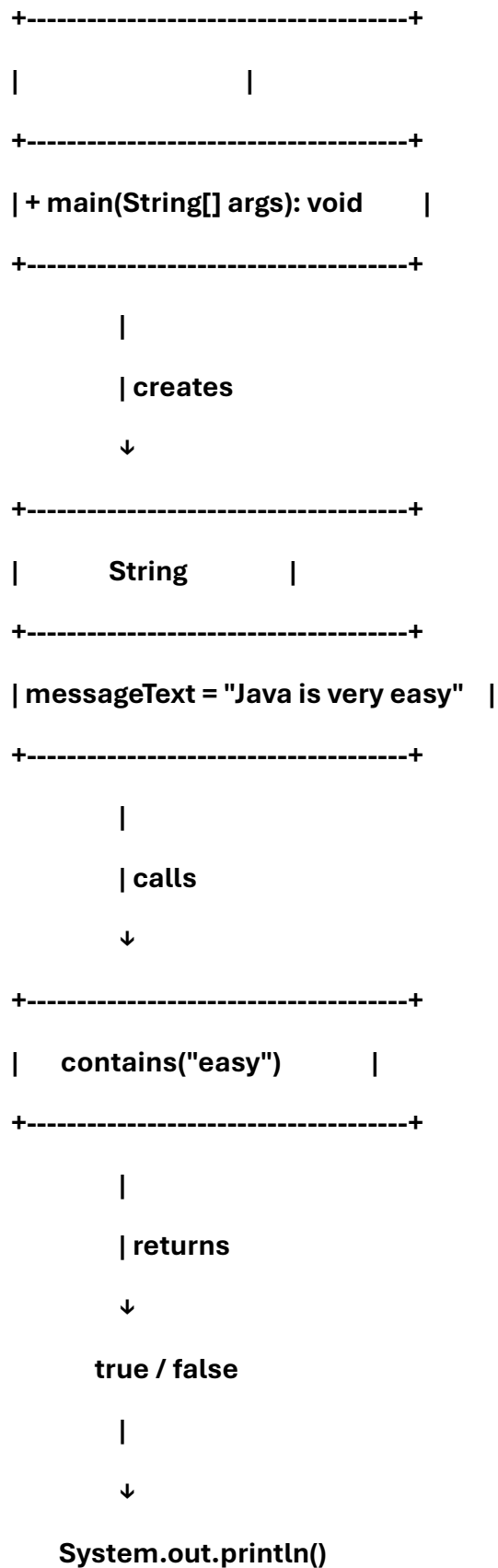
2. Algorithm

Algorithm: To check whether a String contains a specific sequence of characters

1. Start
 2. Declare a String variable messageText.
 3. Assign "Java is very easy" to messageText.
 4. Apply the contains("easy") method to messageText.
 5. Check whether "easy" is present in the given String.
 6. Display the result of the check.
 7. Stop
-

3. UML Diagram

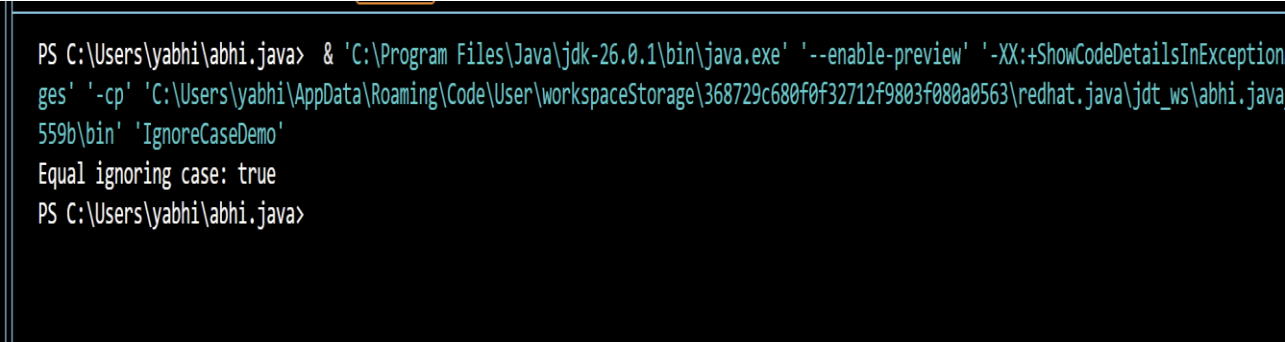




Code :

```
public static void main(String[] args) {  
    String messageText = "Java is very easy";  
  
    System.out.println("Contains 'easy': " + messageText.contains("easy"));  
}  
}
```

Output :



```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInException  
ges' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java  
559b\bin' 'IgnoreCaseDemo'  
Equal ignoring case: true  
PS C:\Users\yabhi\abhi.java>
```

Substring() —

Objective: To understand and demonstrate the Java String method substring() using a simple example.

Task Description: Write and execute the class SubstringDemo to demonstrate that substring() extracts a part of a string using index positions.

Steps

1. Start the program.
 2. Create a class named SubstringDemo.
 3. Define the main() method.
 4. Create a String variable programTitle.
 5. Assign "Programming" to programTitle.
 6. Display the original string.
 7. Use the substring(0, 7) method to extract characters from index 0 to index 6.
 8. Display the extracted substring.
 9. Stop the program.
-

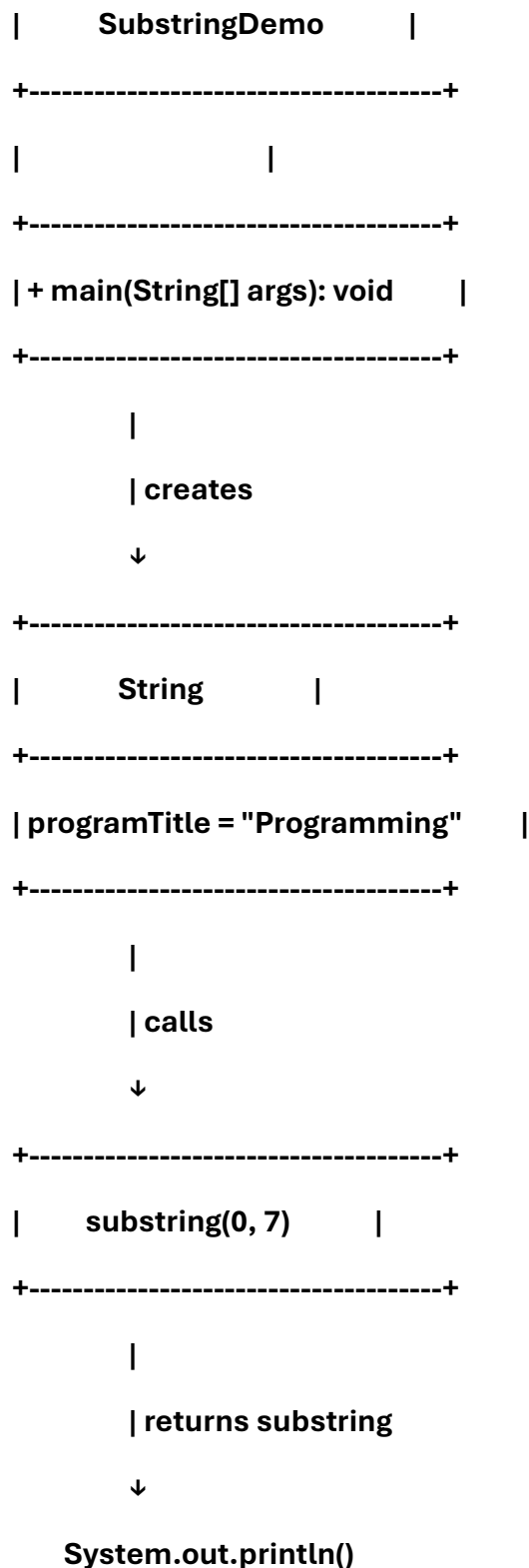
2. Algorithm

Algorithm: To extract a substring from a String

1. Start
 2. Declare a String variable programTitle.
 3. Assign "Programming" to programTitle.
 4. Display the original String.
 5. Apply the substring(0, 7) method to programTitle.
 6. Extract characters starting from index 0 up to, but not including, index 7.
 7. Display the extracted substring.
 8. Stop
-

3. UML Diagram

+-----+



Code :

```
class SubstringDemo {  
    public static void main(String[] args) {  
        String programTitle = "Programming";
```

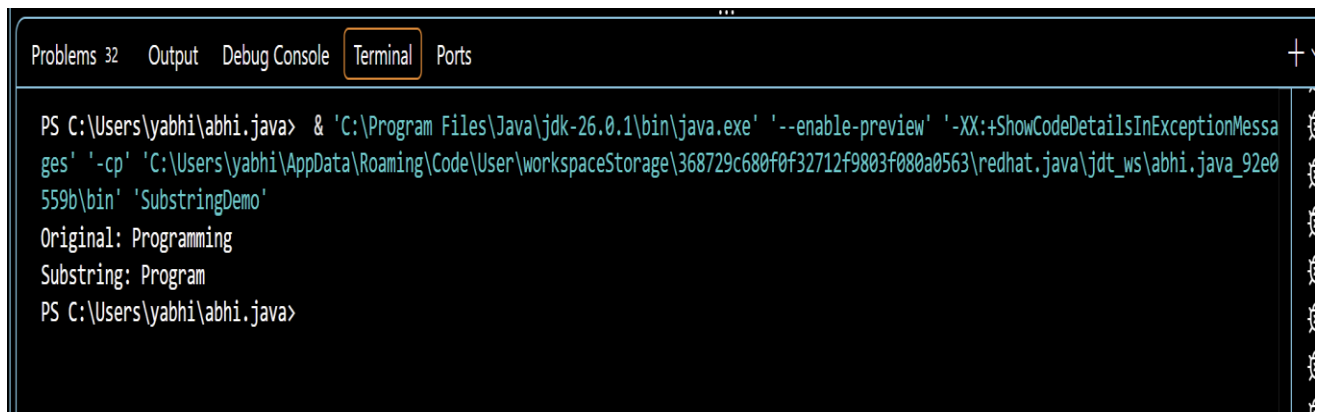
```
    System.out.println("Original: " + programTitle);

    System.out.println("Substring: "+ programTitle.substring(0, 7));

}

}
```

Output :



The screenshot shows a terminal window with a dark background and light-colored text. The terminal title bar includes tabs for 'Problems 32', 'Output', 'Debug Console', 'Terminal' (which is selected and highlighted with an orange border), and 'Ports'. The terminal content shows the command prompt 'PS C:\Users\yabhi\abhi.java>' followed by a long command to run a Java program. The output of the program is displayed in two lines: 'Original: Programming' and 'Substring: Program'. The prompt returns to 'PS C:\Users\yabhi\abhi.java>'.

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'SubstringDemo'
Original: Programming
Substring: Program
PS C:\Users\yabhi\abhi.java>
```

IndexOf () —

Objective: To understand and demonstrate the Java String method `indexOf()` using a simple example.

Task Description: Write and execute the class `IndexOfDemo` to demonstrate that `indexOf()` finds the first position of a specified character or text

Steps

1. Start the program.
 2. Create a class named `StringIndexDemo`.
 3. Define the `main()` method.
 4. Use a try block to write the code that may cause an exception.
 5. Create a String variable `text` and assign "Java".
 6. Use the `charAt(03)` method to access the character at index 3.
 7. If the index is outside the valid range, an exception occurs.
 8. Use the catch block to handle the `StringIndexOutOfBoundsException`.
 9. Display an appropriate exception message.
 10. Stop the program.
-

2. Algorithm

Algorithm: To handle String index out-of-bounds exception

1. Start
 2. Begin the try block.
 3. Declare a String variable `text`.
 4. Assign "Java" to `text`.
 5. Use `charAt(03)` to access a character.
 6. If the index is invalid, catch the `StringIndexOutOfBoundsException`.
 7. Display the exception message.
 8. Stop
-

3. UML Diagram

```

+-----+
|      StringIndexDemo      |
+-----+
|                          |
+-----+
| + main(String[] args): void |
+-----+

    |
    ↓

+-----+
| try   |
+-----+

    |
    ↓

+-----+
| text = "Java"      |
+-----+

    |
    ↓

+-----+
| charAt(03)         |
+-----+

    |
    Exception occurs
    ↓

+-----+
| catch             |
| StringIndexOutOfBounds |

```



System.out.println()

Code :

```
class StringIndexDemo {

    public static void main(String[] args) {

        try {

            String text = "Java";

            System.out.println(text.charAt(03));

        } catch (StringIndexOutOfBoundsException e) {

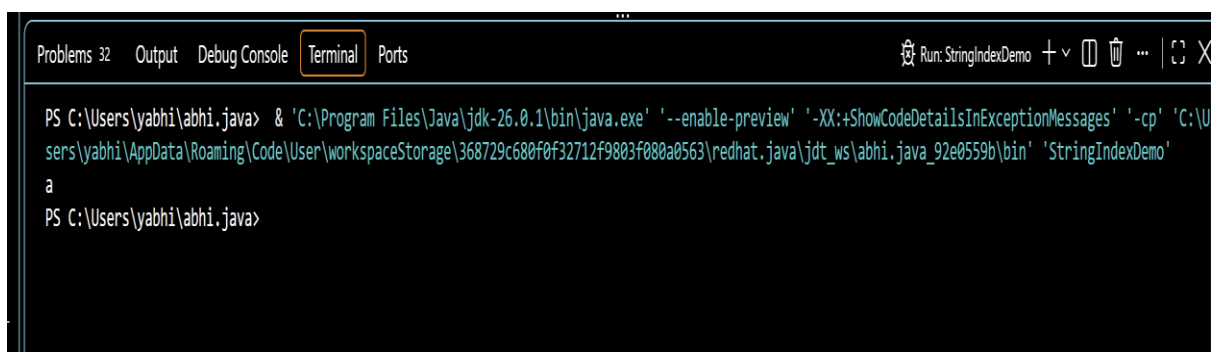
            System.out.println("Exception: String index is out of bounds");

        }

    }

}
```

Output :



```
Problems 32  Output  Debug Console  Terminal  Ports
Run: StringIndexDemo + - [ ] ... [ ] X

PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StringIndexDemo'
a
PS C:\Users\yabhi\abhi.java>
```

LastIndexOf() —

Objective: To understand and demonstrate the Java String method `lastIndexOf()` using a simple example.

Task Description: Write and execute the class `LastIndexDemo` to demonstrate that `lastIndexOf()` finds the last position of a specified character or text.

Steps

1. Start the program.
 2. Create a class named `LastIndexDemo`.
 3. Define the `main()` method.
 4. Create a String variable `languageName`.
 5. Assign "Programming" to `languageName`.
 6. Use the `lastIndexOf('g')` method to find the last occurrence of the character 'g'.
 7. Display the last index of 'g'.
 8. Stop the program.
-

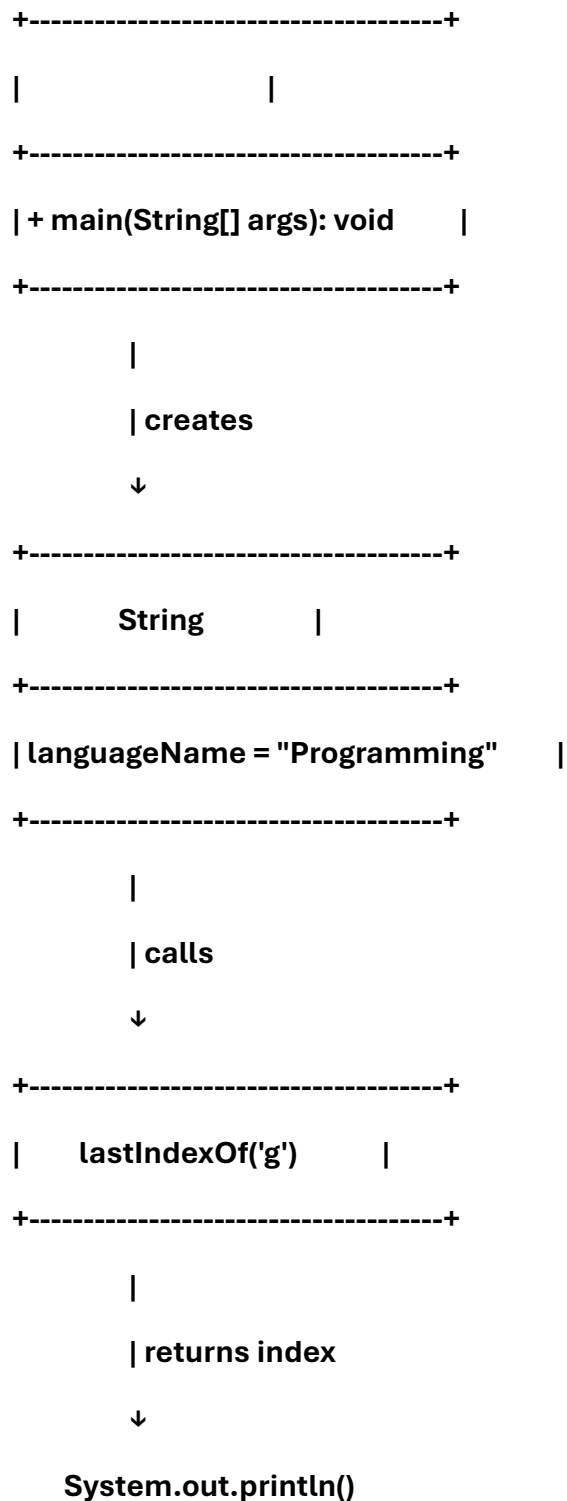
2. Algorithm

Algorithm: To find the last index of a character in a String

1. Start
 2. Declare a String variable `languageName`.
 3. Assign "Programming" to `languageName`.
 4. Apply the `lastIndexOf('g')` method.
 5. Find the last occurrence of the character 'g'.
 6. Display the index of the last occurrence.
 7. Stop
-

3. UML Diagram





Code :

```

class LastIndexDemo {

    public static void main(String[] args) {

        String languageName = "Programming";
    }
}

```

```
        System.out.println("Last index of 'g': "+ languageName.lastIndexOf('g'));
    }
}
```

Output :



The screenshot shows an IDE terminal window with the following content:

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'LastIndexDemo'
Last index of 'g': 10
PS C:\Users\yabhi\abhi.java>
```

The terminal window has a title bar with tabs for 'Problems 32', 'Output', 'Debug Console', 'Terminal' (which is active and highlighted with an orange border), and 'Ports'. On the right side of the title bar, there is a 'Run: LastIndexDemo' button and several icons for window management.

Replace() —

Objective: To understand and demonstrate the Java String method `replace()` using a simple example.

Task Description: Write and execute the class `StringReplaceDemo` to demonstrate that `replace()` replaces characters or a sequence of characters with another sequence.

Steps

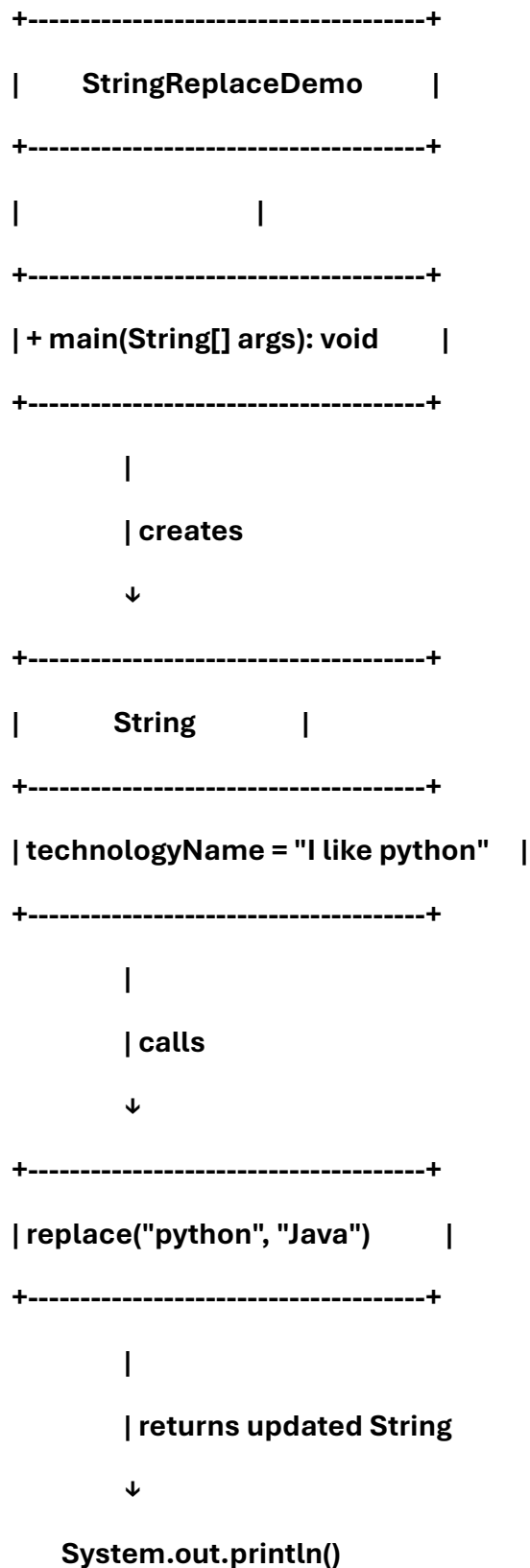
1. Start the program.
 2. Create a class named `StringReplaceDemo`.
 3. Define the `main()` method.
 4. Create a String variable `technologyName`.
 5. Assign "I like python" to `technologyName`.
 6. Display the original String.
 7. Use the `replace()` method to replace "python" with "Java".
 8. Display the updated String.
 9. Stop the program.
-

2. Algorithm

Algorithm: To replace a substring in a String

1. Start
 2. Declare a String variable `technologyName`.
 3. Assign "I like python" to `technologyName`.
 4. Display the original String.
 5. Apply the `replace("python", "Java")` method.
 6. Replace "python" with "Java".
 7. Display the updated String.
 8. Stop
-

3. UML Diagram



Code :

```
class StringReplaceDemo {
    public static void main(String[] args) {
```

```
String technologyName = "I like python";

System.out.println("Original: " + technologyName);

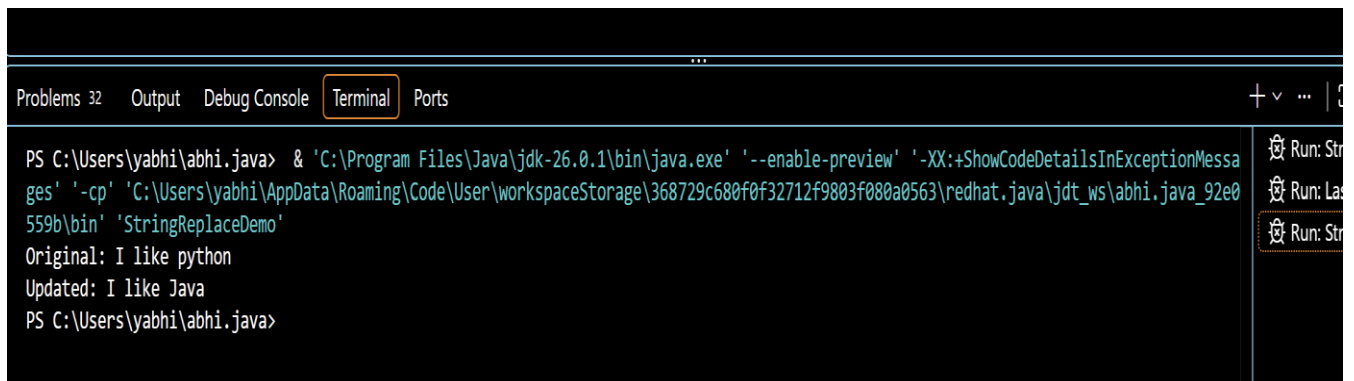
System.out.println("Updated: "

    + technologyName.replace("python", "Java"));

}

}
```

Output :



The screenshot shows a Java IDE interface with a terminal window. The terminal displays the command to run a Java program and its output. The command is: `PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StringReplaceDemo'`. The output is: `Original: I like python`, `Updated: I like Java`, and `PS C:\Users\yabhi\abhi.java>`. The terminal window has tabs for Problems, Output, Debug Console, Terminal, and Ports. The Terminal tab is active. On the right side of the terminal, there are three buttons labeled 'Run: Str', 'Run: La', and 'Run: Str'.

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StringReplaceDemo'
Original: I like python
Updated: I like Java
PS C:\Users\yabhi\abhi.java>
```

Trim() —

Objective: To understand and demonstrate the Java String method trim() using a simple example.

Task Description: Write and execute the class StringTrimDemo to demonstrate that trim() removes leading and trailing whitespace from a string.

Steps

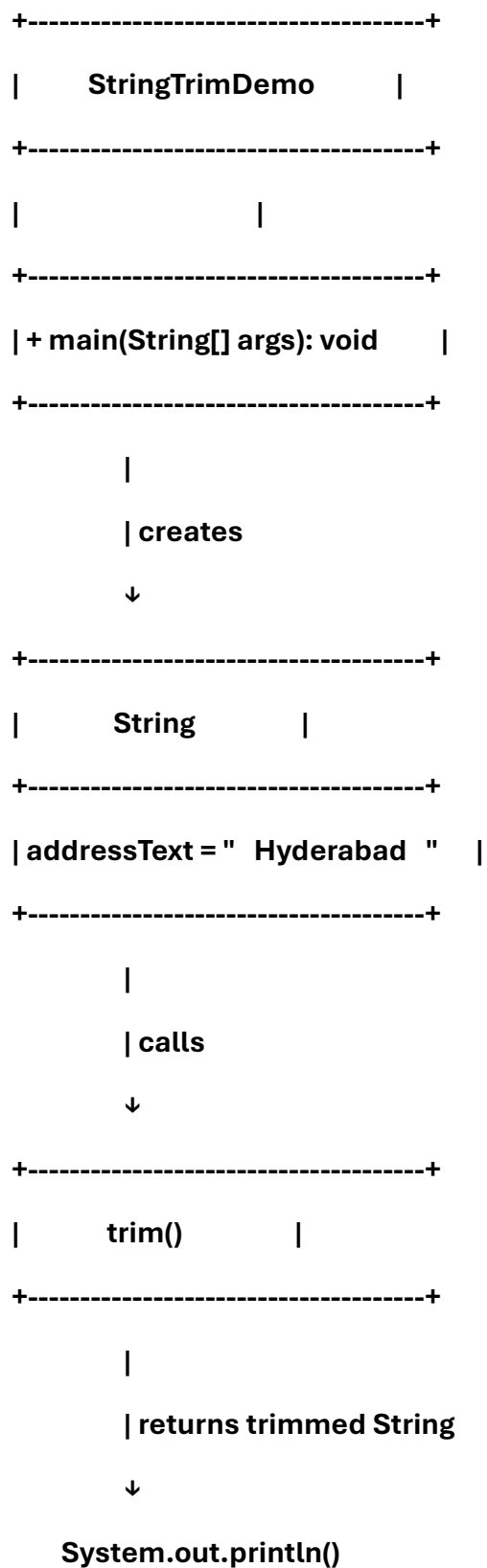
1. Start the program.
 2. Create a class named StringTrimDemo.
 3. Define the main() method.
 4. Create a String variable addressText.
 5. Assign " Hyderabad " to addressText, which contains spaces before and after the text.
 6. Display the String before trimming.
 7. Use the trim() method to remove leading and trailing spaces.
 8. Display the String after trimming.
 9. Stop the program.
-

2. Algorithm

Algorithm: To remove leading and trailing spaces from a String

1. Start
 2. Declare a String variable addressText.
 3. Assign " Hyderabad " to addressText.
 4. Display the original String.
 5. Apply the trim() method to addressText.
 6. Remove the spaces from the beginning and end of the String.
 7. Display the trimmed String.
 8. Stop
-

3. UML Diagram



Code :

```
class StringTrimDemo {
```

```
public static void main(String[] args) {  
    String addressText = "  Hyderabad  ";  
  
    System.out.println("Before: [" + addressText + "]");  
    System.out.println("After: [" + addressText.trim() + "]");  
}  
}
```

Output :

Concat() —

Objective: To understand and demonstrate the Java String method concat() using a simple example.

Task Description: Write and execute the class StringConcatDemo to demonstrate that concat() joins two strings together.

Steps

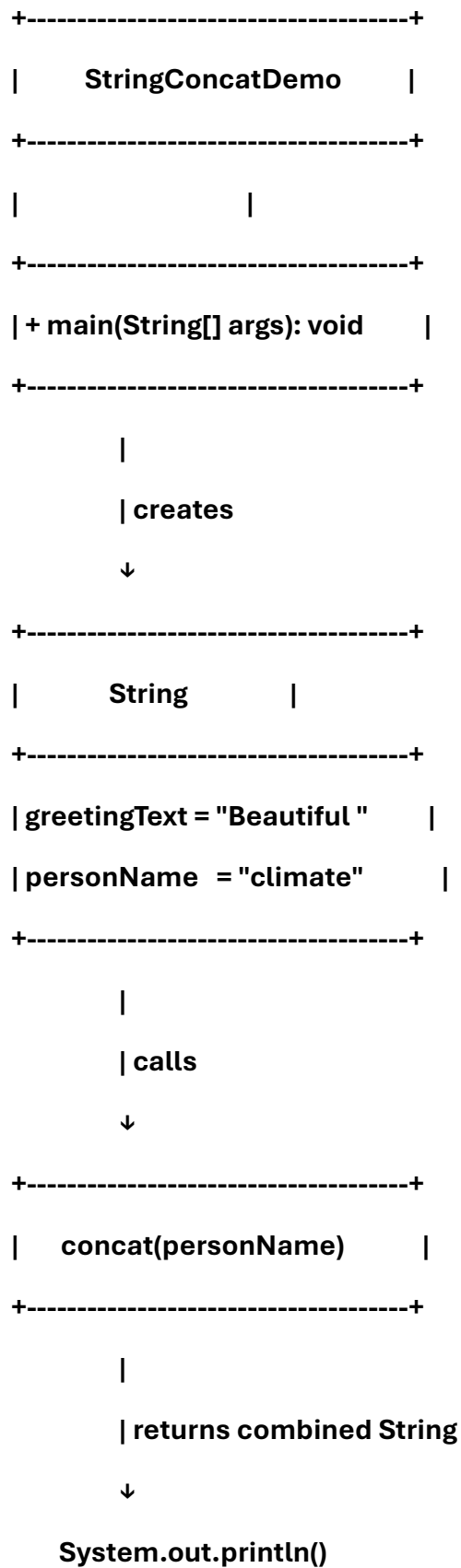
1. Start the program.
 2. Create a class named StringConcatDemo.
 3. Define the main() method.
 4. Create a String variable greetingText.
 5. Assign "Beautiful " to greetingText.
 6. Create another String variable personName.
 7. Assign "climate" to personName.
 8. Use the concat() method to join both strings.
 9. Display the combined String using System.out.println().
 10. Stop the program.
-

2. Algorithm

Algorithm: To concatenate two Strings

1. Start
 2. Declare a String variable greetingText.
 3. Assign "Beautiful " to greetingText.
 4. Declare another String variable personName.
 5. Assign "climate" to personName.
 6. Apply the concat() method to greetingText.
 7. Pass personName as the argument to concat().
 8. Display the combined String.
 9. Stop
-

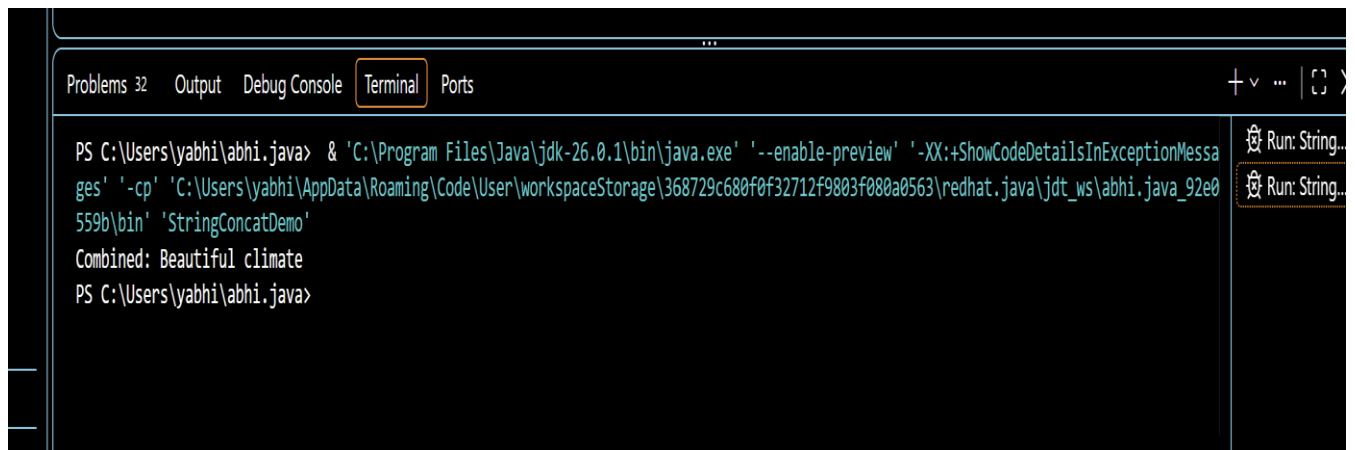
3. UML Diagram



Code :

```
class StringConcatDemo {  
    public static void main(String[] args) {  
        String greetingText = "Beautiful ";  
        String personName = "climate";  
  
        System.out.println("Combined: " + greetingText.concat(personName));  
    }  
}
```

Output :



The screenshot shows an IDE's integrated terminal window. The 'Terminal' tab is active, displaying the command prompt for a PowerShell session. The command executed is to run a Java program named 'StringConcatDemo' with specific JVM flags and a classpath. The output of the program is 'Combined: Beautiful climate'. The terminal window also shows tabs for 'Problems 32', 'Output', 'Debug Console', and 'Ports' at the top. On the right side, there are buttons for 'Run: String...' and 'Run: String...'.

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StringConcatDemo'  
Combined: Beautiful climate  
PS C:\Users\yabhi\abhi.java>
```

Problems 32 Output Debug Console **Terminal** Ports

Run: StringTrimDemo + ▾ □ 🗑 ... | [] X

```
PS C:\Users\yabhi\abhi.java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\yabhi\AppData\Roaming\Code\User\workspaceStorage\368729c680f0f32712f9803f080a0563\redhat.java\jdt_ws\abhi.java_92e0559b\bin' 'StringTrimDemo'
Before: [  Hyderabad  ]
After: [Hyderabad]
PS C:\Users\yabhi\abhi.java>
```